

PC 604 CS — DEEP LEARNING

Question Bank: 2 Marks & 6 Marks Questions with Answers

PC 604 CS — DEEP LEARNING

Question Bank (2 Marks & 6 Marks Questions with Answers)

B.E. AI/ML — Osmania University Prepared with reference to the prescribed textbook: Aston Zhang, Zachary C. Lipton, Mu Li, and Alexander J. Smola, “Dive into Deep Learning” (d2l.ai), and the course syllabus.

UNIT - I: Artificial Neural Networks & Linear Neural Networks

2 Marks Questions

Q1. What is a Perceptron? A perceptron is the simplest artificial neuron model. It takes several binary or real-valued inputs, multiplies each by a weight, sums them along with a bias term, and passes the result through a threshold (step) activation function to produce a binary output (0 or 1). It was designed by Frank Rosenblatt to model a simplified biological neuron and forms the basic building block of neural networks.

Q2. What is the XOR problem, and why is it significant? The XOR (exclusive-OR) problem is the task of classifying points such that the output is 1 only when exactly one of the two binary inputs is 1. It is significant because the XOR function is **not linearly separable** — no single straight line (decision boundary) can separate the two classes in input space. This showed that a single-layer perceptron cannot solve XOR, motivating the development of multilayer networks.

Q3. State the Perceptron Training Rule. The perceptron training rule updates weights whenever a training example is misclassified: $w_i \leftarrow w_i + \eta(y - \hat{y})x_i$ where η is the learning rate, y is the true label, $\hat{y}$ is the predicted output, and x_i is the input. The rule is applied iteratively until the perceptron correctly classifies all (linearly separable) training examples.

Q4. Name any four commonly used activation functions. Sigmoid, Tanh (hyperbolic tangent), ReLU (Rectified Linear Unit), and Softmax are four commonly used activation functions.

Q5. Why do we need non-linear activation functions in neural networks? Without a non-linear activation function, stacking multiple layers is equivalent to a single linear transformation, so the network cannot model complex, non-linear relationships in data. Non-

linear activations allow the network to approximate arbitrary functions and solve problems like XOR that are not linearly separable.

Q6. What is linear regression? Linear regression is a supervised learning technique that models the relationship between input features and a continuous target variable as a weighted linear combination of the features plus a bias term: $\hat{y} = w^T x + b$. The weights are learned by minimizing a loss function, typically mean squared error.

Q7. What is the loss function used in linear regression? The squared error loss (Mean Squared Error) is commonly used: $L = (1/n) \sum (\hat{y}_i - y_i)^2$, which measures the average squared difference between predicted and actual values.

Q8. What is softmax regression used for? Softmax regression (multinomial logistic regression) is used for multi-class classification. It generalizes logistic regression by converting raw output scores (logits) for each class into a probability distribution over all classes using the softmax function, so that the outputs are non-negative and sum to 1.

Q9. Write the softmax function. For a vector of logits $z = (z_1, \dots, z_k)$, softmax is: $\text{softmax}(z_i) = e^{z_i} / \sum_{j=1}^k e^{z_j}$. This converts each logit into a probability proportional to its exponential value.

Q10. What is the Image Classification Dataset commonly used to illustrate softmax regression? The Fashion-MNIST dataset is commonly used. It consists of 60,000 training and 10,000 testing grayscale images (28×28 pixels) across 10 clothing categories (e.g., T-shirt, trouser, sneaker), and serves as a standard benchmark for simple classification models like softmax regression.

Q11. What is the cross-entropy loss? Cross-entropy loss measures the difference between the predicted probability distribution and the true (one-hot) label distribution. For a true label y and predicted probability $\hat{y}$, it is $L = - \sum y_i \log(\hat{y}_i)$. It is the standard loss function for classification tasks, including softmax regression.

6 Marks Questions

Q1. Explain the structure of a Perceptron and derive/explain the Perceptron Training Rule with an example.

Structure: A perceptron takes an input vector $x = (x_1, x_2, \dots, x_n)$, each associated with a weight w_i , plus a bias b . It computes a weighted sum: $z = \sum w_i x_i + b$ and applies a step activation function: $\hat{y} = 1$ if $z \geq 0$, else 0.

Training Rule: The perceptron learns by adjusting weights only when it makes a mistake. For each training example (x, y) : 1. Compute the prediction $\hat{y}$ using current weights. 2. If $\hat{y} = y$, no update is made. 3. If $\hat{y} \neq y$, update weights: $w_i \leftarrow w_i + \eta(y - \hat{y})x_i$, and $b \leftarrow b + \eta(y - \hat{y})$.

Example: Suppose we want to learn the AND function with inputs $(0,0) \rightarrow 0$, $(0,1) \rightarrow 0$, $(1,0) \rightarrow 0$, $(1,1) \rightarrow 1$. Starting with weights $w_1 = w_2 = 0$, $b = 0$, and $\eta = 0.1$, the perceptron processes each example, and whenever the prediction is wrong, it nudges the weights in the direction that

would have produced the correct output. After several passes (“epochs”) through the data, the weights converge to values such as $w_1 = w_2 = 0.1$, $b = -0.1$, which correctly separates the AND function with a straight line. The Perceptron Convergence Theorem guarantees that if the data is linearly separable, this rule will find a separating hyperplane in a finite number of steps. However, for the XOR function, no such line exists, so the algorithm never converges — this is exactly why the XOR problem exposed the perceptron’s limitation and motivated multilayer networks.

Q2. Explain the XOR problem and show why a single perceptron fails to solve it.

XOR is defined as: output = 1 if exactly one input is 1, else 0. The truth table is (0,0)→0, (0,1)→1, (1,0)→1, (1,1)→0.

A single perceptron computes a decision boundary that is a straight line (hyperplane) in input space: $w_1x_1 + w_2x_2 + b = 0$. For XOR, the two positive points (0,1) and (1,0) and the two negative points (0,0) and (1,1) are arranged so that any line separating one pair of classes necessarily misclassifies the other. Geometrically, the positive and negative classes lie on opposite diagonals of the unit square, and no single line can separate diagonal corners of a square from the other diagonal corners — this is the formal meaning of “not linearly separable.”

Resolution: The problem can be solved by using a multilayer perceptron (MLP) with a hidden layer. For instance, two hidden units can each implement simple linear boundaries (such as OR and NAND), and an output unit can combine them with an AND-like operation, effectively computing $\text{XOR} = (x_1 \text{ OR } x_2) \text{ AND NOT}(x_1 \text{ AND } x_2)$. This demonstrates that stacking layers with non-linear activation functions allows a network to represent functions that a single perceptron cannot, which historically justified the shift from single-layer to multilayer (deep) neural networks.

Q3. Explain any four activation functions used in neural networks, along with their mathematical formulas, ranges, and use cases.

1. **Sigmoid:** $\sigma(x) = 1 / (1 + e^{-x})$. Range (0, 1). Used in binary classification output layers and traditionally in hidden layers. Drawback: suffers from vanishing gradients for large $|x|$.
2. **Tanh (hyperbolic tangent):** $\tanh(x) = (e^x - e^{-x}) / (e^x + e^{-x})$. Range (-1, 1). Zero-centered, so it often converges faster than sigmoid, but still suffers from vanishing gradients at extremes.
3. **ReLU (Rectified Linear Unit):** $\text{ReLU}(x) = \max(0, x)$. Range $[0, \infty)$. Computationally efficient, mitigates vanishing gradient problem for positive inputs, and is the default choice in most modern deep networks. Drawback: “dying ReLU” — neurons can get stuck outputting zero if inputs are always negative.
4. **Softmax:** $\text{softmax}(z_i) = e^{z_i} / \sum_j e^{z_j}$. Used in the output layer of multi-class classification to convert logits into a probability distribution summing to 1.

Each activation function introduces non-linearity, which is essential for the network to learn complex decision boundaries; the choice of activation affects training speed, gradient flow, and the type of task (regression vs. classification) the network is suited for.

Q4. Explain Linear Regression and derive the objective (loss) function used to train it.

Linear regression assumes a linear relationship between the input features $x = (x_1, \dots, x_d)$ and a continuous target y : $\hat{y} = w_1x_1 + w_2x_2 + \dots + w_dx_d + b = w^T x + b$

Given a training dataset of n examples, we want to choose weights w and bias b that make predictions $\hat{y}_i$ as close as possible to the true values y_i . The most common measure of “closeness” is the squared error for each example: $l^{(i)}(w, b) = (1/2)(\hat{y}_i - y_i)^2$. The overall loss over the dataset is the average: $L(w, b) = (1/n) \sum_{i=1}^n (1/2)(w^T x_i + b - y_i)^2$

This loss is minimized using an optimization algorithm — most commonly (mini-batch) stochastic gradient descent — which iteratively updates w and b in the direction that reduces the loss: $w \leftarrow w - \eta \nabla_w L$, $b \leftarrow b - \eta \nabla_b L$ where η is the learning rate. For simple linear regression, a closed-form solution also exists (the normal equations), but gradient-based optimization is preferred in deep learning because it generalizes to more complex models where closed-form solutions do not exist.

Q5. Explain Softmax Regression with its architecture, loss function, and a worked numerical example.

Softmax regression extends linear regression to multi-class classification. Given an input x , the model computes one linear score (logit) per class: $o_j = w_j^T x + b_j$, for classes $j = 1$ to k . These logits are converted to probabilities using the softmax function: $\hat{y}_j = e^{o_j} / \sum_{l=1}^k e^{o_l}$ so that all $\hat{y}_j$ are non-negative and sum to 1, making them interpretable as class probabilities.

Loss function: Cross-entropy loss is used to compare the predicted probability vector $\hat{y}$ against the true one-hot label vector y : $L = - \sum_j y_j \log(\hat{y}_j)$. Since y is one-hot (1 for the true class, 0 elsewhere), this simplifies to $L = -\log(\hat{y}_{\text{true class}})$, meaning the loss penalizes low predicted probability for the correct class.

Worked example: Suppose a 3-class classifier produces logits $o = (2.0, 1.0, 0.1)$ for an image whose true label is class 1. Softmax probabilities are approximately $\hat{y} = (0.659, 0.242, 0.099)$. Since the true class is class 1 with predicted probability 0.659, the loss is $L = -\log(0.659) \approx 0.417$. Training adjusts the weights via gradient descent to increase the predicted probability of the correct class over many examples, typically evaluated on a dataset such as Fashion-MNIST for image classification.

UNIT – II: Multilayer Perceptrons & Optimization Algorithms

2 Marks Questions

Q1. What is a Multilayer Perceptron (MLP)? An MLP is a feedforward neural network consisting of an input layer, one or more hidden layers, and an output layer, where each hidden layer applies a linear transformation followed by a non-linear activation function. Stacking

hidden layers allows the network to learn complex, non-linear mappings that a single-layer perceptron cannot represent.

Q2. Define underfitting and overfitting. Underfitting occurs when a model is too simple to capture the underlying pattern in the data, resulting in high training and test error. Overfitting occurs when a model learns the training data too well, including noise, resulting in low training error but high test error (poor generalization).

Q3. What is weight decay? Weight decay is a regularization technique that adds a penalty term (typically the L2 norm of the weights, $\lambda \|w\|^2$) to the loss function, discouraging large weight values and thus reducing overfitting by keeping the model simpler.

Q4. What is Dropout? Dropout is a regularization technique where, during training, a random subset of neurons (with probability p) is temporarily “dropped” (set to zero) in each forward pass, forcing the network to not rely on any single neuron and improving generalization.

Q5. Define Forward Propagation and Backward Propagation. Forward propagation is the process of computing the output of a neural network by passing input data sequentially through each layer. Backward propagation (backpropagation) is the process of computing gradients of the loss with respect to each weight by applying the chain rule backward from the output layer to the input layer, used to update weights during training.

Q6. What is a computational graph? A computational graph is a directed graph representation of a sequence of mathematical operations, where nodes represent variables or operations and edges represent dependencies. It is used to systematically apply the chain rule during backpropagation.

Q7. What is the vanishing/exploding gradient problem? It refers to gradients becoming extremely small (vanishing) or extremely large (exploding) as they are propagated backward through many layers, making training slow or unstable. Careful weight initialization (e.g., Xavier initialization) and activation function choice help mitigate this.

Q8. What is Gradient Descent? Gradient Descent is an iterative optimization algorithm that updates parameters in the direction opposite to the gradient of the loss function to minimize it: $\theta \leftarrow \theta - \eta \nabla L(\theta)$, where η is the learning rate.

Q9. Differentiate between Batch Gradient Descent, Stochastic Gradient Descent (SGD), and Mini-batch SGD. Batch gradient descent computes the gradient using the entire dataset per update (accurate but slow). SGD computes the gradient using a single randomly chosen example per update (fast but noisy). Mini-batch SGD computes the gradient using a small batch of examples per update, balancing accuracy and computational efficiency — it is the most commonly used variant in practice.

Q10. What is Momentum in optimization? Momentum accelerates gradient descent by accumulating a moving average of past gradients and using it to update parameters, which helps dampen oscillations and speeds up convergence, especially in ravines of the loss surface: $v \leftarrow \beta v + \nabla L$, $\theta \leftarrow \theta - \eta v$.

Q11. What is Adagrad? Adagrad adapts the learning rate for each parameter individually based on the historical sum of squared gradients, giving smaller updates to frequently updated

parameters and larger updates to infrequently updated ones. A drawback is that the learning rate can shrink too aggressively over time.

Q12. What is RMSProp and how does it improve on Adagrad? RMSProp modifies Adagrad by using an exponentially decaying moving average of squared gradients instead of accumulating all past gradients, preventing the learning rate from shrinking too much and allowing better performance on non-convex problems.

Q13. What is the Adam optimizer? Adam (Adaptive Moment Estimation) combines the ideas of Momentum and RMSProp: it maintains exponentially decaying averages of both the gradient (first moment) and the squared gradient (second moment), and uses bias-corrected estimates to compute adaptive per-parameter learning rates. It is widely used as a default optimizer in deep learning.

Q14. What is Learning Rate Scheduling? Learning rate scheduling refers to strategies for changing the learning rate during training (e.g., decreasing it over time, step decay, cosine annealing), which helps achieve faster convergence early in training and finer convergence near the optimum later.

6 Marks Questions

Q1. Explain the architecture of a Multilayer Perceptron and describe forward and backward propagation with a computational graph.

An MLP consists of an input layer, one or more hidden layers, and an output layer. Each layer l computes: $z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$, $a^{(l)} = \phi(z^{(l)})$ where ϕ is a non-linear activation function (e.g., ReLU), W and b are the layer's weights and bias, and $a^{(0)} = x$ is the input.

Forward propagation computes activations layer by layer from input to output, ultimately producing the prediction $\hat{y}$ and the loss $L(\hat{y}, y)$.

Backward propagation computes gradients of L with respect to every weight using the chain rule, working backward from the output layer. For example, at the output layer: $\partial L / \partial z^{(L)}$ is computed directly, then $\partial L / \partial W^{(L)} = \partial L / \partial z^{(L)} \cdot a^{(L-1)T}$. This gradient is propagated to the previous layer: $\partial L / \partial a^{(L-1)} = W^{(L)T} \partial L / \partial z^{(L)}$, and multiplied by the local derivative $\phi'(z^{(L-1)})$ to continue backward.

A *computational graph* organizes these operations as nodes (variables/operations) and edges (dependencies), letting frameworks like automatic differentiation systems apply the chain rule mechanically and efficiently, reusing intermediate values computed during the forward pass. This systematic backward traversal is what allows deep networks with many layers to be trained efficiently via gradient descent.

Q2. Explain the problem of overfitting and underfitting with the help of the bias-variance tradeoff. How can Weight Decay and Dropout be used to address overfitting?

Underfitting happens when the model's capacity is too low relative to the complexity of the task — the model has high bias, performing poorly on both training and test data (e.g., fitting a straight line to clearly non-linear data).

Overfitting happens when the model has excess capacity and starts to memorize noise and idiosyncrasies of the training set — the model has high variance, achieving very low training error but high test error, since it fails to generalize to unseen data.

The bias-variance tradeoff describes this balance: simple models have high bias/low variance, while complex models have low bias/high variance. Model selection (e.g., choosing the right number of layers/hidden units, using a validation set) aims to find the sweet spot that minimizes total generalization error.

Weight Decay (L2 regularization): Adds a penalty $\lambda \|w\|^2$ to the loss function: $L_{\text{reg}} = L + \lambda \|w\|^2$. This discourages the model from assigning very large weights to any feature, effectively simplifying the model and reducing variance, at a controlled cost to bias, since λ controls the strength of regularization.

Dropout: During training, each neuron is retained with probability $(1-p)$ and dropped (set to 0) with probability p , independently, on every forward pass. This prevents the network from relying too heavily on any single neuron and forces it to learn more robust, distributed representations. At test time, all neurons are used, but their outputs are typically scaled to match the expected value seen during training. Both techniques reduce overfitting by constraining model complexity without changing the network architecture itself.

Q3. Explain the significance of proper weight initialization and numerical stability in training deep networks.

As gradients are propagated backward through many layers via the chain rule, they are computed as products of many partial derivatives. If these values are consistently greater than 1, the product can grow exponentially with depth (exploding gradients); if consistently less than 1, the product can shrink toward zero (vanishing gradients). Both scenarios make training unstable or extremely slow, since weight updates become either too large (causing divergence) or too small (causing the network to stop learning, especially in earlier layers).

Proper weight initialization addresses this by ensuring that the variance of activations and gradients remains roughly constant across layers at the start of training. **Xavier (Glorot) initialization**, for instance, sets initial weights by drawing from a distribution with variance inversely proportional to the number of input (and output) units of a layer, so that signal variance is preserved as it passes forward and backward through the network. Additional practices such as using ReLU-family activations (which have gradient 1 for positive inputs, avoiding saturation), batch normalization, and careful learning-rate selection also directly support numerical stability, allowing very deep networks to be trained reliably.

Q4. Explain Gradient Descent, Stochastic Gradient Descent, and Mini-batch SGD with their update rules, advantages and disadvantages.

Gradient Descent (Batch): Update rule: $\theta \leftarrow \theta - \eta (1/n) \sum_{i=1}^n \nabla L_i(\theta)$. Uses the entire dataset to compute one gradient step. Advantage: stable, accurate gradient direction.

Disadvantage: computationally expensive per step for large datasets, memory-intensive.

Stochastic Gradient Descent (SGD): Update rule: $\theta \leftarrow \theta - \eta \nabla L_i(\theta)$ for a randomly chosen single example i . Advantage: very fast per-update, can escape shallow local minima due to noise. Disadvantage: noisy convergence path, may not settle precisely at the minimum.

Mini-batch SGD: Update rule: $\theta \leftarrow \theta - \eta (1/|B|) \sum_{i \in B} \nabla L_i(\theta)$, where B is a small randomly sampled batch (e.g., 32, 64, 128 examples). Advantage: balances the stability of batch GD with the speed of SGD, and takes advantage of vectorized/parallel hardware (GPUs) for efficient computation. This is the standard method used in practically all deep learning training.

Convexity is discussed because gradient descent is guaranteed to converge to the global minimum only for convex loss surfaces; deep neural network losses are generally non-convex, so in practice these algorithms converge to good local minima or saddle points that empirically generalize well, aided by variants like Momentum and Adam.

Q5. Explain the Momentum, Adagrad, RMSProp, and Adam optimization algorithms with their update equations, and compare their strengths.

Momentum: $v_t = \beta v_{t-1} + \nabla L(\theta_{t-1})$; $\theta_t = \theta_{t-1} - \eta v_t$. It accumulates a “velocity” of past gradients, smoothing the path and speeding convergence through narrow ravines, damping oscillations.

Adagrad: Maintains $G_t = G_{t-1} + (\nabla L)^2$, and updates $\theta_t = \theta_{t-1} - (\eta / \sqrt{G_t + \epsilon}) \nabla L$. Parameters with large historical gradients get smaller updates, adapting the learning rate per-parameter; it is well-suited for sparse data, but the ever-growing G_t causes the effective learning rate to shrink too much over time, stalling training.

RMSProp: Fixes Adagrad’s decay problem using an exponential moving average: $G_t = \gamma G_{t-1} + (1-\gamma)(\nabla L)^2$, $\theta_t = \theta_{t-1} - (\eta / \sqrt{G_t + \epsilon}) \nabla L$. This keeps the accumulated squared-gradient estimate “fresh,” preventing the learning rate from vanishing, and works well on non-stationary/non-convex objectives typical of deep learning.

Adam: Combines Momentum and RMSProp — it tracks both the first moment (mean of gradients) $m_t = \beta_1 m_{t-1} + (1-\beta_1) \nabla L$ and second moment (mean of squared gradients) $v_t = \beta_2 v_{t-1} + (1-\beta_2)(\nabla L)^2$, applies bias correction (since $m_0, v_0 = 0$), and updates $\theta_t = \theta_{t-1} - \eta \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$. Adam typically converges quickly and robustly with little hyperparameter tuning, making it the most popular default optimizer in modern deep learning, though plain SGD with momentum and a well-tuned learning rate schedule sometimes generalizes slightly better for some tasks.

UNIT – III: Convolutional Neural Networks

2 Marks Questions

Q1. What is a Convolutional Neural Network (CNN)? A CNN is a type of deep neural network specifically designed for processing grid-like data such as images. It uses convolutional

layers with learnable filters (kernels) that slide over the input to detect local spatial patterns like edges, textures, and shapes, exploiting spatial locality and parameter sharing.

Q2. What is a kernel/filter in CNNs? A kernel (or filter) is a small matrix of learnable weights that slides (convolves) across the input, computing dot products with local regions to produce a feature map that highlights specific patterns such as edges or textures.

Q3. What are the key principles behind CNNs? The key principles are: (1) local connectivity/receptive fields — each neuron only connects to a small local region of the input; (2) parameter sharing — the same filter weights are applied across all spatial positions, drastically reducing the number of parameters; and (3) translation invariance — features are detected regardless of their exact location in the image.

Q4. Why are multiple filters used in a convolutional layer? Multiple filters allow the network to detect multiple different features (e.g., horizontal edges, vertical edges, colors, textures) simultaneously at each layer, producing multiple feature maps that are stacked to form a richer representation of the input.

Q5. What is AlexNet, and why was it significant? AlexNet is a deep CNN architecture (2012) with 8 layers (5 convolutional + 3 fully connected) that won the ImageNet competition by a large margin. It was significant because it demonstrated that deep CNNs trained on GPUs with ReLU activations and dropout could dramatically outperform traditional computer vision methods, sparking the modern deep learning revolution.

Q6. What is VGG, and what is its key design principle? VGG is a CNN architecture that uses a uniform design of small 3×3 convolutional filters stacked in blocks, followed by max-pooling, repeated multiple times with increasing channel depth. Its key insight is that network depth (many small-filter layers) is more important for accuracy than using larger filters.

Q7. What is Network in Network (NiN)? NiN replaces the large fully connected layers at the end of a CNN with 1×1 convolutions (acting like per-pixel fully connected mini-networks) and uses global average pooling before the output, drastically reducing the number of parameters compared to architectures like AlexNet and VGG.

Q8. What is GoogLeNet's key innovation? GoogLeNet introduces the “Inception module,” which applies multiple convolutions of different kernel sizes (e.g., 1×1, 3×3, 5×5) and pooling in parallel within the same layer and concatenates their outputs, allowing the network to capture features at multiple scales efficiently.

Q9. What is Batch Normalization? Batch Normalization is a technique that normalizes the inputs of each layer (across a mini-batch) to have zero mean and unit variance, then applies a learnable scale and shift. It stabilizes and accelerates training by reducing internal covariate shift and allows higher learning rates.

Q10. What is a Residual Network (ResNet) and what problem does it solve? ResNet introduces “skip connections” (residual connections) that add the input of a block directly to its output: $y = F(x) + x$. This solves the degradation/vanishing gradient problem in very deep networks by allowing gradients to flow directly through the shortcut path, enabling the training of networks with hundreds of layers.

Q11. What is DenseNet? DenseNet (Densely Connected Network) connects each layer to every other layer in a feed-forward fashion within a “dense block” — each layer receives, as input, the concatenated feature maps of all preceding layers. This encourages feature reuse and improves gradient flow, often with fewer parameters than ResNet.

6 Marks Questions

Q1. Explain the working principle of a CNN, including convolution operation, kernels, and multiple filters, with a suitable example.

A CNN processes an image by sliding small learnable filters (kernels), e.g., of size 3×3 , across the input. At each position, the filter computes an element-wise multiplication with the corresponding local patch of the input followed by a summation, producing a single output value in the resulting feature map. Mathematically, for input X and kernel K : $(X * K)[i,j] = \sum_m \sum_n X[i+m, j+n] \cdot K[m,n]$

Example: Consider a 5×5 grayscale image patch and a 3×3 edge-detection kernel such as $\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$. Sliding this kernel over the image highlights vertical edges by producing large positive/negative values where pixel intensity changes sharply horizontally, and near-zero values in flat regions.

A single convolutional layer typically applies **multiple filters** in parallel (e.g., 32 or 64 different kernels), each producing its own feature map; these are stacked along the channel dimension to give a rich, multi-channel representation. Stacking several convolutional layers (often with pooling layers in between to reduce spatial resolution) allows the network to build up a hierarchy of features — early layers detect edges and colors, middle layers detect textures and shapes, and deeper layers detect complex object parts. This exploits local connectivity (each neuron looks only at a small receptive field) and parameter sharing (the same kernel weights are reused at every spatial position), which drastically reduces the number of parameters compared to a fully connected network and makes CNNs translation-invariant and well-suited for image data.

Q2. Describe the architecture of AlexNet and explain why it was a breakthrough for deep learning.

AlexNet (Krizhevsky et al., 2012) consists of 5 convolutional layers followed by 3 fully connected layers, totaling around 60 million parameters. Its key architectural features include:

- Large first-layer filters (11×11) with stride 4 to quickly reduce spatial dimensions on high-resolution (224×224) ImageNet images, followed by smaller filters in later layers.
- Use of the **ReLU** activation function instead of sigmoid/tanh, which sped up training considerably by avoiding vanishing gradients for positive inputs.
- **Overlapping max-pooling** layers to down-sample feature maps.
- **Dropout** in the fully connected layers to combat overfitting, given the large number of parameters.
- Training on **GPUs**, which made it computationally feasible to train such a large network on the large-scale ImageNet dataset.

AlexNet’s breakthrough significance lies in its dramatic reduction of the ImageNet top-5 error rate compared to traditional shallow computer-vision pipelines (handcrafted features + SVM),

by a margin of more than 10 percentage points. This proved that deep, end-to-end learned representations combined with sufficient data, compute (GPUs), and good regularization/activation choices could vastly outperform earlier approaches, directly triggering the modern deep learning era in computer vision.

Q3. Explain the VGG architecture and its design principle of using blocks of small convolutional filters.

VGG networks are built from repeated **VGG blocks**, each consisting of several 3×3 convolutional layers (with padding to preserve spatial size) followed by a 2×2 max-pooling layer that halves the spatial resolution. A typical VGG-16 network stacks 5 such blocks with increasing numbers of channels (e.g., $64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 512$), followed by fully connected layers and a softmax output.

The key design insight is that using several stacked 3×3 filters achieves the same effective receptive field as a single larger filter (e.g., two 3×3 layers $\approx$ one 5×5 receptive field) while using fewer parameters and adding an extra non-linearity (ReLU) between them, which improves the network's expressive power. VGG demonstrated empirically that increasing network **depth**, using a simple and uniform block structure, was a more effective way to improve accuracy than tuning filter sizes per layer — a principle that strongly influenced later architectures. The main drawback of VGG is its large number of parameters (mostly in the fully connected layers), making it memory- and computation-heavy compared to later architectures like NiN or GoogLeNet.

Q4. Compare Network in Network (NiN), GoogLeNet (Inception), and explain how they improve computational efficiency over VGG/AlexNet.

NiN: Replaces the large, parameter-heavy fully connected layers used at the end of AlexNet/VGG with **1×1 convolutions**, which act like a small fully connected network applied independently at every spatial location, allowing more non-linear combination of channels without a large parameter cost. NiN also uses **global average pooling** over the final feature maps (instead of flattening followed by dense layers) to directly produce one value per class, eliminating the huge number of parameters normally found in the classifier head.

GoogLeNet (Inception): Introduces the **Inception module**, which runs several convolutions of different kernel sizes (1×1 , 3×3 , 5×5) and a pooling operation in parallel on the same input, then concatenates their output feature maps along the channel dimension. To keep computation manageable, 1×1 convolutions are used before the larger $3 \times 3/5 \times 5$ convolutions to reduce the number of channels (“bottleneck”). This lets the network capture multi-scale features (fine details and broader context) at each layer while controlling parameter/computation cost, and GoogLeNet achieves this with far fewer parameters than VGG despite being deeper.

Comparison: Both NiN and GoogLeNet target the same weakness in AlexNet/VGG — the huge number of parameters concentrated in fully connected layers or in using only single-scale convolutions — but NiN solves it via 1×1 convolutions + global average pooling, while GoogLeNet solves it via parallel multi-scale Inception modules with dimensionality-reducing bottlenecks. Both achieve competitive or better accuracy with significantly reduced parameter counts and computational cost compared to VGG.

Q5. Explain Batch Normalization, Residual Networks (ResNet), and Densely Connected Networks (DenseNet), highlighting the specific training problem each solves.

Batch Normalization: During training, the distribution of layer inputs can shift as the parameters of previous layers change (internal covariate shift), which slows training and requires careful, small learning rates. Batch Normalization addresses this by normalizing each layer's input over a mini-batch — subtracting the batch mean and dividing by the batch standard deviation — then applying learnable scale (γ) and shift (β) parameters: $\hat{x} = (x - \mu_B) / \sqrt{(\sigma_B^2 + \epsilon)}$, $y = \gamma\hat{x} + \beta$. This stabilizes the distribution of inputs to each layer, allows the use of higher learning rates, reduces sensitivity to initialization, and has a mild regularizing effect.

ResNet: As networks become very deep, training accuracy can paradoxically degrade because gradients vanish or the optimization landscape becomes harder to navigate (the “degradation problem”). ResNet solves this using **residual (skip) connections**: instead of learning a direct mapping $H(x)$, each block learns a residual $F(x) = H(x) - x$, and the block's output is $F(x) + x$. The identity shortcut allows gradients to flow directly backward through the addition operation unimpeded, making it possible to train networks with 50, 101, or even 152+ layers successfully, since in the worst case a block can simply learn $F(x) \approx 0$ and behave like an identity mapping.

DenseNet: Extends the idea of skip connections by connecting **each layer within a dense block to every subsequent layer**, with each layer receiving the concatenation of all preceding layers' feature maps as input (rather than an element-wise sum, as in ResNet). This maximizes information and gradient flow throughout the network, strongly encourages feature reuse (reducing redundant learning), and often results in more parameter-efficient networks than ResNet at similar or better accuracy, though at higher memory cost during training due to the growing concatenated feature maps.

UNIT – IV: Recurrent Neural Networks

2 Marks Questions

Q1. What is a sequence model? A sequence model is a model designed to handle input or output data that comes as an ordered sequence (e.g., text, time series, speech), where the order of elements matters and where predictions typically depend on previous elements in the sequence.

Q2. What is Text Preprocessing in the context of RNNs? Text preprocessing includes steps such as tokenization (splitting text into words/characters/subwords), building a vocabulary, and converting tokens into numerical indices (or embeddings) so that text can be fed into a neural network.

Q3. What is a language model? A language model estimates the probability of a sequence of words (or the probability of the next word given previous words): $P(w_1, w_2, \dots, w_T) = \prod_t$

$P(w_t | w_1, \dots, w_{t-1})$). It is fundamental to tasks like text generation, machine translation, and speech recognition.

Q4. Write the basic recurrence relation of an RNN. $h_t = \varphi(W_{hh} h_{t-1} + W_{xh} x_t + b_h)$ where h_t is the hidden state at time t , x_t is the input at time t , and φ is an activation function (typically $\tanh$). The hidden state carries information from previous time steps forward.

Q5. What is Backpropagation Through Time (BPTT)? BPTT is the algorithm used to train RNNs; it “unrolls” the recurrent network across all time steps into an equivalent feedforward computational graph and applies standard backpropagation across this unrolled graph to compute gradients with respect to the shared weights at every time step.

Q6. What problem do vanilla RNNs face with long sequences, and why? Vanilla RNNs suffer from vanishing/exploding gradients over long sequences because the gradient computed via BPTT involves repeated multiplication by the same weight matrix (and derivative of the activation) across many time steps, causing gradients to shrink toward zero or grow unboundedly, which makes it hard to learn long-range dependencies.

Q7. What is a Gated Recurrent Unit (GRU)? A GRU is a variant of RNN that uses two gates — a reset gate and an update gate — to control how much of the past hidden state to keep versus how much to update with new information, helping the network better capture long-term dependencies while using fewer parameters than an LSTM.

Q8. What is a Long Short-Term Memory (LSTM) network? An LSTM is an RNN variant that introduces a memory cell along with three gates (input, forget, and output gates) that regulate the flow of information into, out of, and within the cell, allowing the network to retain or discard information over long sequences and effectively mitigate the vanishing gradient problem.

Q9. What is a Deep RNN? A Deep RNN stacks multiple RNN layers on top of each other, where the hidden state output of one layer at each time step becomes the input to the next layer, allowing the network to learn more abstract, hierarchical temporal representations.

Q10. What is a Bidirectional RNN? A Bidirectional RNN processes the input sequence in both forward and backward directions using two separate hidden state sequences, and combines them, allowing predictions at each time step to depend on both past and future context — useful for tasks like text classification but not for real-time prediction tasks.

Q11. What is the Encoder-Decoder architecture? The Encoder-Decoder architecture consists of an encoder network that compresses an input sequence into a fixed-length context vector (or set of hidden states), and a decoder network that generates an output sequence conditioned on that context — commonly used for sequence-to-sequence tasks such as machine translation.

Q12. What is Beam Search? Beam search is a heuristic search algorithm used during decoding in sequence generation tasks; instead of keeping only the single most probable next token (greedy search) or exploring every possibility (exhaustive search), it keeps track of the top- k (“beam width” k) most probable partial sequences at each step, offering a practical trade-off between output quality and computational cost.

6 Marks Questions

Q1. Explain the architecture of a basic RNN and describe Backpropagation Through Time (BPTT) for training it.

A Recurrent Neural Network processes sequential data $x_1, x_2, \dots, x_T$ by maintaining a hidden state h_t that is updated at each time step using both the current input and the previous hidden state: $h_t = \varphi(W_{\{xh\}}x_t + W_{\{hh\}}h_{t-1} + b_h)$ $o_t = W_{\{hq\}}h_t + b_q$ Here, the same weight matrices ($W_{\{xh\}}$, $W_{\{hh\}}$, $W_{\{hq\}}$) are shared across all time steps, which allows the model to handle sequences of variable length and to generalize temporal patterns learned at one position in the sequence to other positions.

To train an RNN, we cannot apply standard backpropagation directly because of the recurrent (cyclic) dependency between hidden states. **BPTT** resolves this by conceptually “unrolling” the RNN across time into a long feedforward computational graph, where each time step is a distinct layer that shares the same weights. The loss is typically the sum (or average) of per-time-step losses: $L = \sum_t l(o_t, y_t)$. Gradients are then computed by applying the chain rule backward through this unrolled graph, accumulating the gradient contribution from every time step for the shared weight matrices, e.g.: $\partial L / \partial W_{\{hh\}} = \sum_t \partial l_t / \partial W_{\{hh\}}$ (accounting for the dependency of h_t on all previous $h_{t'}$) Because the same $W_{\{hh\}}$ is multiplied repeatedly across many time steps in this chain, the gradient can vanish or explode for long sequences — this is why techniques like gradient clipping, truncated BPTT (only unrolling a limited number of steps), and gated architectures (GRU/LSTM) are used in practice.

Q2. Explain the vanishing gradient problem in RNNs and describe how LSTM solves it using its gate mechanism.

In a vanilla RNN, the gradient of the loss with respect to an early hidden state involves a product of many Jacobian terms (one per time step), each roughly proportional to $W_{\{hh\}}$ and $\varphi'(\cdot)$. If the dominant eigenvalues of this product are less than 1, the gradient shrinks exponentially with sequence length (vanishing gradient), making it nearly impossible for the network to learn dependencies that span many time steps; if greater than 1, gradients explode.

LSTM solves this by introducing a separate **cell state** C_t that is updated additively rather than through repeated multiplicative transformations, controlled by three sigmoid **gates**: - **Forget gate**: $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$ — decides what fraction of the old cell state to keep. - **Input gate**: $i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$ — decides how much new information to add. - **Candidate memory**: $\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C)$ - **Cell state update**: $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$ - **Output gate**: $o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$, $h_t = o_t \odot \tanh(C_t)$

Because the cell state update is largely additive ($f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$), gradients can flow backward through many time steps largely unchanged when the forget gate stays close to 1, effectively creating a “gradient highway” that avoids the repeated multiplicative shrinking seen in vanilla RNNs. This allows LSTMs to learn dependencies across much longer sequences than plain RNNs.

Q3. Explain the Gated Recurrent Unit (GRU) architecture and compare it with LSTM.

A GRU simplifies the LSTM by merging the cell state and hidden state into a single hidden state h_t , and using only two gates: - **Reset gate**: $r_t = \sigma(W_r[h_{t-1}, x_t] + b_r)$ — controls how much of the previous hidden state to “forget” when computing the candidate hidden state. - **Update gate**: $z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$ — controls the balance between retaining the previous hidden state and using the new candidate. - **Candidate hidden state**: $\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h)$ - **Final hidden state**: $h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t$

Comparison with LSTM: GRU has fewer parameters (2 gates and one state vs. 3 gates and two states in LSTM), so it trains faster and requires less memory, and often performs comparably to LSTM on many tasks, especially with smaller datasets. LSTM’s separate cell state and additional output gate give it slightly more expressive control over what is remembered/exposed, which can help on tasks requiring very long-term memory. In practice, the choice between them is often empirical — GRU is a common lightweight default, while LSTM remains a strong choice when maximal modeling capacity for long dependencies is needed.

Q4. Explain the Encoder-Decoder architecture and the Sequence-to-Sequence (Seq2Seq) model with an example application such as machine translation.

The **Encoder-Decoder** architecture is designed for tasks where both the input and the output are sequences, potentially of different lengths (e.g., translating an English sentence to French). The **encoder** (typically an RNN, GRU, or LSTM) reads the input sequence $x_1, \dots, x_T$ token by token and produces a final hidden state (context vector) that summarizes the entire input sequence: $h_T = \text{Encoder}(x_1, \dots, x_T)$ The **decoder** (another RNN) is initialized with this context vector and generates the output sequence one token at a time, where each generated token is fed back as input for generating the next token: $s_t = \text{Decoder}(s_{t-1}, y_{t-1}, h_T)$, $\hat{y}_t = \text{softmax}(W s_t)$ Training uses “teacher forcing,” where the true previous target token y_{t-1} (rather than the model’s own prediction) is fed to the decoder during training to stabilize learning, while at inference time the model’s own previous prediction is used.

Example — Machine Translation: For translating the English sentence “I am a student” into French, the encoder processes the English tokens sequentially, producing a context vector summarizing the sentence’s meaning. The decoder then generates French tokens one at a time (“Je”, “suis”, “étudiant”) conditioned on this context, until it produces a special end-of-sequence token. The Machine Translation dataset typically used consists of paired sentences in the source and target languages, tokenized and padded/truncated to fixed lengths for batch training. A key limitation of the basic Encoder-Decoder model is that compressing an entire (potentially long) sentence into a single fixed-length context vector can lose information, which later architectures address using attention mechanisms.

Q5. Explain Beam Search decoding and compare it with Greedy Search, illustrating with an example.

During sequence generation (e.g., machine translation), at each decoding time step the model outputs a probability distribution over the vocabulary for the next token, and we must choose a strategy for selecting tokens to construct a complete output sequence.

Greedy Search simply picks the single highest-probability token at each step and moves on. It is fast ($O(1)$ candidates per step) but can be suboptimal overall — a locally optimal choice at

one step can lead to a poor overall sequence, because greedy search never reconsiders earlier choices.

Exhaustive Search would consider every possible sequence and pick the one with the highest overall probability, guaranteeing the optimal answer, but this is computationally infeasible since the number of possible sequences grows exponentially with vocabulary size and sequence length ($|V|^T$).

Beam Search offers a practical middle ground: at each time step, it retains the top-k partial sequences (where k is the “beam width”) ranked by their cumulative log-probability so far, rather than just the single best one. At the next step, each of the k sequences is expanded with every possible next token, and again only the top-k highest-scoring sequences (out of $k \times |V|$ candidates) are kept. This continues until an end-of-sequence token is generated for enough candidates, and the highest-scoring complete sequence among the beam is returned.

Example: Suppose $k = 2$ and at the first decoding step the top-2 tokens are “Je” (probability 0.5) and “Il” (probability 0.3). Greedy search would commit to “Je” alone. Beam search instead keeps both “Je” and “Il” and expands both at the next step — perhaps “Je suis” ($0.5 \times 0.4 = 0.20$) and “Il est” ($0.3 \times 0.6 = 0.18$) — retaining the better-scoring continuations even if the second-best first token ultimately leads to a better full sentence. This allows beam search to often find higher-quality (higher joint-probability) sequences than greedy search, at a computational cost that scales linearly with beam width k rather than exponentially, making it the standard decoding strategy in practical sequence-to-sequence systems.

UNIT – V: Autoencoders & Generative Adversarial Networks

2 Marks Questions

Q1. What is an Autoencoder? An autoencoder is an unsupervised neural network that learns to compress input data into a lower-dimensional latent representation (encoding) and then reconstruct the original input from that representation (decoding), trained by minimizing the reconstruction error between the input and output.

Q2. Name different types of autoencoders. Common types include: Vanilla (basic) autoencoders, Sparse autoencoders, Denoising autoencoders, Convolutional autoencoders, and Variational Autoencoders (VAEs).

Q3. What is a Denoising Autoencoder? A denoising autoencoder is trained to reconstruct the original, clean input from a corrupted (noisy) version of it, forcing the network to learn robust features that capture the true underlying structure of the data rather than memorizing noise.

Q4. What is a Variational Autoencoder (VAE)? A VAE is a generative variant of the autoencoder that learns to encode inputs into a probability distribution (typically Gaussian, parameterized by a mean and variance) in the latent space rather than a single fixed point, enabling it to generate new, realistic data samples by sampling from the latent distribution.

Q5. State two applications of autoencoders. Two applications are: (1) dimensionality reduction / feature learning for downstream tasks, and (2) anomaly detection, where high reconstruction error signals an unusual or anomalous input. Other applications include denoising, image compression, and generative modeling.

Q6. What is a Generative Adversarial Network (GAN)? A GAN is a generative model consisting of two neural networks — a Generator that creates fake data from random noise, and a Discriminator that tries to distinguish real data from generated (fake) data — trained together in an adversarial (minimax) game until the generator produces realistic samples.

Q7. Write the objective function of a GAN. The minimax objective is: $\min_G \max_D V(D, G) = E_{x \sim p_{\text{data}}} [\log D(x)] + E_{z \sim p_z} [\log(1 - D(G(z)))]$ The discriminator D tries to maximize this (correctly classify real vs. fake), while the generator G tries to minimize it (fool the discriminator).

Q8. What is a DCGAN (Deep Convolutional GAN)? DCGAN is a GAN architecture that uses convolutional and transposed-convolutional (deconvolutional) layers instead of fully connected layers in the generator and discriminator, along with batch normalization and specific activation choices (ReLU/LeakyReLU, Tanh output), making it well-suited for generating realistic images.

Q9. What is “mode collapse” in GAN training? Mode collapse occurs when the generator learns to produce only a limited variety of outputs (or even a single output) that reliably fool the discriminator, failing to capture the full diversity of the true data distribution.

6 Marks Questions

Q1. Explain the architecture of an Autoencoder and describe its training process with a mathematical formulation.

An autoencoder consists of two parts: - **Encoder:** a function f that maps the input $x \in \mathbb{R}^d$ to a lower-dimensional latent representation $z = f(x) \in \mathbb{R}^k$, where $k < d$, typically implemented as one or more neural network layers. - **Decoder:** a function g that reconstructs the input from the latent code: $\hat{x} = g(z) = g(f(x))$.

The network is trained end-to-end (encoder and decoder jointly) by minimizing a **reconstruction loss**, most commonly the mean squared error between the input and its reconstruction: $L(x, \hat{x}) = \|x - \hat{x}\|^2 = \|x - g(f(x))\|^2$ Since there is no external label — the “target” is the input itself — this is a form of **unsupervised/self-supervised learning**. Because the latent dimension k is smaller than the input dimension d (a “bottleneck”), the network is forced to learn a compressed representation that captures the most important, informative features of the data while discarding redundant or noisy details, rather than simply learning the identity function. Once trained, the encoder alone can be used to extract compact feature representations for downstream tasks (dimensionality reduction, visualization, or as input to a classifier), while both encoder and decoder together can be used for tasks like denoising or anomaly detection, where a large reconstruction error indicates an input dissimilar from the training distribution.

Q2. Explain different types of Autoencoders (Sparse, Denoising, Convolutional, Variational) and their specific applications.

- **Sparse Autoencoder:** Adds a sparsity penalty (e.g., an L1 penalty on the activations of the latent layer, or a KL-divergence penalty encouraging most latent units to be near zero) to the reconstruction loss. This forces the network to represent the input using only a small number of “active” latent units at a time, encouraging it to learn meaningful, disentangled features even when the latent dimension is not smaller than the input — useful for feature learning.
- **Denoising Autoencoder (DAE):** Trained by corrupting the input x with noise (e.g., Gaussian noise or randomly zeroing some pixels) to get $\tilde{x}$, then training the network to reconstruct the original clean x from $\tilde{x}$: $L = \|x - g(f(\tilde{x}))\|^2$. This forces the model to learn robust, generalizable features rather than trivially copying input to output, and is directly useful for image denoising applications.
- **Convolutional Autoencoder:** Replaces the fully connected layers of a vanilla autoencoder with convolutional layers in the encoder and transposed-convolutional (or upsampling + convolution) layers in the decoder. This preserves spatial structure and is much more effective for image data, used in applications like image compression and image inpainting.
- **Variational Autoencoder (VAE):** Instead of mapping input to a single fixed latent vector, the encoder outputs the parameters (mean μ and variance σ^2) of a probability distribution over the latent space, and the latent code z is sampled from this distribution (using the “reparameterization trick” to allow gradients to flow through the sampling step). The loss combines reconstruction error with a KL-divergence term that regularizes the latent distribution to stay close to a standard normal distribution. This structured latent space allows VAEs to be used as true **generative models** — new realistic samples can be generated by sampling z from the prior and decoding it — unlike vanilla autoencoders, which do not guarantee a well-structured latent space.

Q3. Explain the architecture and training procedure of a Generative Adversarial Network (GAN) with its objective function.

A GAN consists of two competing neural networks trained simultaneously: - **Generator (G):** Takes a random noise vector z (sampled from a simple prior distribution, e.g., Gaussian or uniform) and maps it to a synthetic data sample $G(z)$, attempting to make it indistinguishable from real data. - **Discriminator (D):** Takes a data sample (real or generated) and outputs the probability that it is real, $D(x) \in [0,1]$.

They are trained via a **minimax game**: $\min_G \max_D V(D,G) = E_{\{x \sim p_{\text{data}}\}}[\log D(x)] + E_{\{z \sim p_z\}}[\log(1 - D(G(z)))]$

Training alternates between two steps: 1. **Train D:** Update D's weights to maximize its ability to correctly classify real samples as real ($D(x) \rightarrow 1$) and generated samples as fake ($D(G(z)) \rightarrow 0$), by performing gradient ascent on $V(D,G)$. 2. **Train G:** Update G's weights to make the discriminator misclassify generated samples as real ($D(G(z)) \rightarrow 1$), by performing gradient

descent on $\log(1 - D(G(z)))$ (or equivalently, gradient ascent on $\log D(G(z))$, a common practical modification that provides stronger gradients early in training).

Ideally, this adversarial process converges to a **Nash equilibrium** where the generator produces samples so realistic that the discriminator can no longer distinguish them from real data better than random guessing ($D(x) = 0.5$ everywhere). In practice, GAN training can be unstable and prone to issues such as mode collapse (the generator produces limited variety) and vanishing gradients for the generator when the discriminator becomes too strong too quickly, which motivate practical tricks such as label smoothing, alternative loss functions, and careful learning-rate/architecture balancing between G and D.

Q4. Explain the DCGAN (Deep Convolutional GAN) architecture and how it improves upon a vanilla GAN for image generation.

A vanilla GAN typically uses fully connected layers for both generator and discriminator, which works reasonably for low-dimensional or simple data but performs poorly on high-dimensional image data, since fully connected layers ignore spatial structure and require enormous numbers of parameters.

DCGAN improves image generation quality and training stability through several key architectural guidelines: 1. **All-convolutional architecture**: The discriminator uses strided convolutional layers to progressively downsample the image into a single real/fake score, while the generator uses transposed convolutional (fractionally-strided) layers to progressively upsample a low-dimensional noise vector z into a full-resolution image, preserving and exploiting the spatial structure of images. 2. **Batch Normalization**: Applied in both the generator and discriminator (except at the output layer of G and input layer of D) to stabilize training and prevent mode collapse. 3. **Activation functions**: ReLU activations are used in the generator's hidden layers, with a Tanh activation at the output layer (to match the typical $[-1, 1]$ normalized pixel range); LeakyReLU is used throughout the discriminator to avoid the “dying” gradients that plain ReLU can cause in a network intended to always provide useful gradients back to the generator. 4. **No fully connected/pooling layers** (except at the very start/end), relying entirely on strided/transposed convolutions for spatial resizing.

Example application: DCGAN can be trained on a dataset of face images; after training, sampling different random noise vectors z from the prior and passing them through the trained generator produces novel, realistic-looking face images that were not present in the training set, and interpolating smoothly between two noise vectors produces a smooth morph between the corresponding generated images — demonstrating that the network has learned a meaningful, continuous latent representation of the image manifold.

Q5. Compare Autoencoders and GANs as generative/representation-learning models, highlighting their differences in training objective and typical use cases.

Aspect	Autoencoder	GAN
Components	Single encoder + decoder	Two competing networks: Generator + Discriminator
Training objective	Minimize reconstruction loss (e.g., MSE) between input	Adversarial minimax game between G and D

Aspect	Autoencoder	GAN
Supervision signal	and output Self-supervised — input acts as its own target	Discriminator provides an implicit, learned signal of “realism”
Latent space	Deterministic (vanilla AE) or distributional (VAE); not guaranteed to be smooth/generative unless variational	Learned implicitly; not directly observed unless explicitly probed, but typically smooth and semantically meaningful
Primary strength	Compression, denoising, anomaly detection, feature learning	Generating highly realistic, sharp synthetic samples
Typical weakness	Reconstructions from vanilla AEs can be blurry, and vanilla AEs are not designed to generate genuinely novel/realistic samples	Training instability, mode collapse, harder to evaluate quantitatively
Common use cases	Dimensionality reduction, denoising, anomaly detection, pretraining	Image synthesis, style transfer, data augmentation, super-resolution

In summary, autoencoders are primarily representation-learning models optimized for faithful reconstruction (with VAEs bridging toward generative modeling via a probabilistic latent space), while GANs are primarily generative models optimized directly for sample realism through adversarial competition rather than explicit reconstruction, which is why GAN-generated images are often sharper and more realistic than vanilla-autoencoder reconstructions, at the cost of a more difficult and less stable training process.

End of Question Bank — Prepared as per Osmania University B.E. AI/ML Deep Learning (PC 604 CS) syllabus, with reference to “Dive into Deep Learning” (d2l.ai) and Goodfellow, Bengio & Courville’s “Deep Learning” (MIT Press, 2016).